

Feature-Delivery — Arbeitsvorgehensweise

Spezifikation → Planungs-Loop (Lean / Strong) → Implementierungs-Loop → Closure

1 Spezifikation

optional vorgelagert · Ziel: DoD-ready Spec

ado (optional) load → analyse → save · liefert story.md / task.md als Ausgangsbasis

buddy-agent Sparrings-Partner · klärt, verdichtet, prüft Repo, handoff

intake → compress → repo-check → diskussion → plan-prompt

✓ Fertige Spezifikation — klare ACs, keine offenen Fragen, DoD-ready

Abhängigkeiten: acceptance-design software-design-principles

acceptance-design

→ Spezifikation

WAS testen — Anforderungen auf Testbarkeit prüfen

- ACs gegen Prüfkatalog abgleichen
- Untestbare Kriterien schärfen oder Rückfrage markieren
- Output: Testname + AAA-Stichpunkte + Status (F1)
- acceptance-design-agent · standalone · kein Auto-Trigger

★ software-design-principles

→ Spec · Plan · Impl

Qualitätsnordstern — automatisch aktiv in allen Phasen

- sauber · funktional · getestet · wartbar · nachhaltig
- Flow Design · IODA/IOSP · SOLID · Clean Code
- YAGNI · DRY · KISS · keine Verschachtelung
- 2 dedizierte Reviewer-Agents (Opus) in Plan + Impl

test-design

→ Scribes (Impl)

WIE testen — Konventionen für Scribes

- MethodName_Situation_ErwartetesErgebnis
- Arrange / Act / Assert
- .NET: xUnit v3 · FluentAssertions · Moq
- Angular: Karma · Jasmine · TestBed

dev-tooling — MCP Gateway

→ Scouts · Gates

Lesen/Bauen → dev-mcp · Analysieren → codebase-analyzer
· Logs → build-log-filter

dev-mcp

49 Tools · stdio
Filesystem · Build/Test · Scaffolding · Git · Patch · Inspect · Lint

codebase-analyzer

43 Tools · Node · Port 5052
index · review_git_diff · analyze_iosp · find_in_index · Domain-Finder

build-log-filter

Docker · Port 8089
ng serve · Shell-Logs verdichten · Shell-Fallback

FEATURE-DELIVERY — PLANUNGS-LOOP

2a Planung

Modus wird durch Trigger-Wort aktiviert

LEAN Default — ohne Zusatz | STRONG explizit „strong“ | CHECK / CHECK+ Bewertung 1–7 → STOPP, kein Planning

LEAN Default — ohne Zusatz · schnell

Plan-Orchestrator (Opus)

Plant + prüft + reviewed solo — alles in einem Turn, kein Sub-Agent-Fan-out

- ✗ Kein Scout (Phase 3)
- ✗ Kein Topic-Planer (Phase 4b)
- ✗ Kein Review-Subagent-Loop

Für: klaren Scope · bekannte Codebasis · überschaubare Integration

STRONG Volles Arsenal · hohe Konfidenz

Plan-Orchestrator (Opus)

PHASE 3 — SCOUTS:

Scout x1–10 (Sonnet) Code + Test-Coverage kartieren · MCP: codebase-analyzer + dev-mcp

PHASE 4B — TOPIC-PLANER:

Topic-Planer x1–10 (Sonnet) Teilpläne + Akzeptanz→Test-Liste je Topic (§8/F1)

PLAN-REVIEW-LOOP (MAX. 5 ITER.):

Guard Risk Readiness Craft Auditor Design-Principles

Plan-Fixer (Opus) iteratives Patchen · bei Blocker: Mini-Re-Planning

Für: viele Unbekannte · Cross-Service · hohes Risiko · komplexe Integration

Beide Modi — immer Pflicht:

§8/F1 Akzeptanzliste (Testname + AAA-Stichpunkte) · \$UA Uncertainty Audit · requests/plans/plan-*.md persistieren

Nordstern: software-design-principles

2b delivery-inspection — Plan

Plan als Deliverable · universell einsetzbar

6 REVIEWER PARALLEL — ITERATIVER LOOP (MAX. 10 RUNDEN):

Revisor

Anforderungs-Buchhalter · 1:1 Mapping Anforderung → Plan

Skeptiker

Lückenjäger · Top-3 schlimmste Lücken

Normalo

Pragmatische Abnahme · sofort umsetzbar?

Dolmetscher

Fehlinterpretationen · stille Entscheidungen aufdecken

Auftraggeber

Finale Abnahme · würde ich das unterschreiben?

Querdenker

YAGNI-Wächter · Scope-Creep · Zuviel / Zuwenig

↓

Sauber

\$UA leer → Auto-Handoff zur Implementierung

↻

Findings

→ Plan-Orchestrator (Opus) · eindeutig fixbar oder gebündelte User-Rückfrage · iteriert bis sauber

Sauber — \$UA leer · Auto-Handoff zur Implementierung

FEATURE-DELIVERY — IMPLEMENTIERUNGS-LOOP

3a Implementierung

Test-First (§8) · max. 5 Fix-Runden

Impl-Loop-Orchestrator (Opus) Hard Gate (14 Readiness-Fragen) · Integration-Checkpoint · Slice-Coverage-Check · delegiert alles

SCRIBES (TEST-FIRST: RED → GREEN):

Scribe x1–10 (Sonnet) R1–3 Tests nach Akzeptanzliste (Red) → Implementierung (Green) · nur slice-scoped · BoyScout danach

Scribe x1–10 (Opus) R4–5 Eskalation bei hartnäckigen Findings

QUALITY GATES (INTEGRATIONSWEIT · VIA DEV-TOOLING):

G1

Build
dev-mcp

G2

Lint · Inspect · review_git_diff · IOSP
dev-mcp + codebase-analyzer

G3

Design-Principles Review
impl-review-agent (Opus)

G4

Test-Suite
dev-mcp

7 IMPL-REVIEWER PARALLEL (READONLY):

Risk Design-Principles Verifier Readiness Craft Auditor Guard Verifier: fachliche Korrektheit + AC-Map §8/F4 · review_git_diff als Evidenz

Fix-Planer (Opus) → Fix-Scribes → Gates erneut · nach R5: Final-Gate + Rest-Findings-Bericht

Referenz-Skills: test-design software-design-principles angular-developer code-intel-workflow

6 REVIEWER PARALLEL — ITERATIVER LOOP (MAX. 10 RUNDEN):

Revisor Anforderungs-Buchhalter · 1:1 Mapping	Skeptiker Lückenjäger · was fehlt oder halbfertig?	Normalo Pragmatische Abnahme · produktiv einsetzbar?
Dolmetscher Fehlinterpretationen · stille Entscheidungen	Auftraggeber Finale Abnahme · abnahmefähig?	Querdenker YAGNI-Wächter · Scope-Creep



Sauber

Alle 6 Perspektiven ohne Findings → Closure



Findings

→ Impl-Loop-Orchestrator · Fix-Scribe oder gebündelte User-Rückfrage



Sauber — Delivery-Inspection abgeschlossen

4 — CLOSURE

✓ Fertig

Plan-Alignment · Gate-Matrix pro Stack/Runde · 7 Reviews je Runde · Fix-Planer-Evidenz · MCP-Compliance · Rest-Findings-Bericht wenn nötig